

WAX Constitutional Runtime

The repository as it actually exists after the OMEGA cycle: the four tracked authority defects fixed in code, memory, context, objectives and delivery verified end to end, and every remaining boundary documented honestly. 655 tests passing, 13 of 13 live probe checks passing over real HTTP.

Verification 655 tests · 13/13 probe · migrations validated

Violations 14 corrected across two audit cycles · none open

Date September 14, 2026

Table of Contents

1. Executive Summary	2
2. Philosophy: Infrastructure, Not Intelligence	2
3. Architecture and the Runtime Graph	3
4. Memory: The Living Substrate	4
4.1 How memory evolves	4
4.2 Forgetting, stated honestly	4
5. Context: The Model's View of Reality	4
6. Objectives and Continuity	5
7. Agentic Execution, Work and Time	5
8. Capabilities and Environments	5
8.1 The one shared gate	6
8.2 Environments	6
9. Interfaces: Adapters, Not Walls	6
10. Security: The Runtime Enforces	7
11. Evaluation: How We Know	7
12. What Changed in the OMEGA Cycle	8
13. Current Limitations and Honest Boundaries	9
14. Future Expansion Boundaries	9

1. Executive Summary

This report tells you, in plain language, what the WAX runtime actually is today, what was repaired during the OMEGA cycle, and what remains honestly unfinished. Every statement was checked against the code, the test suite, the database migrations, and a live probe that talks to a running system over real HTTP. Nothing here is copied from an earlier report; where earlier numbers were wrong, the repository won.

655	13/13	14	0	21
tests passing	live probe checks	violations corrected	open violations	capabilities

The OMEGA mission asked for one coordinated operation: audit reality, reconcile the architecture with the constitution (*infrastructure, not intelligence*), repair violations, and document everything honestly. The headline result is that the four architectural defects you asked to be tracked until fixed (the duplicated approval creation path, the duplicated approval gate, delivery records written by only one source, and background approvals that could stay invisible to the human) are now fixed in code with tests. They were not closed with documentation.

Beyond those four, the cycle fixed two concurrency races in the message idempotency machine, put every destructive background sweep under leader election so two servers cannot destroy the same directory at once, wired a conversation lifecycle mechanism that claimed to run but never did, and corrected nine comments and schema notes that promised behavior the code did not perform. Three new architectural decision records capture the boundaries that need your decision or a future cycle rather than silent invention: retention policy, the open capability registry, and multi-server state.

2. Philosophy: Infrastructure, Not Intelligence

WAX is built around one architectural constraint, not a slogan: the runtime provides mechanisms, and the intelligence decides what to do with them. In practice this means the code base contains no domain features. There is no study mode, no reminder feature, no birthday manager, no exam tracker. A reminder, when a person needs one, is a composition: durable work scheduled with a time wake, which invokes the generic messaging capability, whose delivery failure becomes recoverable state. The runtime executes that composition without knowing or caring what it was for.

The flip side of that restraint is enforcement. The AI has agency but never sovereignty: it cannot grant itself permissions (the AI role in the authority system is deliberately empty), it cannot declare an objective succeeded while durable work is outstanding (the database decides), and it cannot message a third party (the recipient must be the human's own verified credential). Every one of those statements is enforced by code or by a database constraint, and each has a test behind it.

The test every mechanism must pass: could this mechanism serve many unrelated legitimate objectives without knowing the domain? If yes, it may be infrastructure. If no, it is probably a hardcoded application feature in disguise.

3. Architecture and the Runtime Graph

WAX is a Python runtime of 122 source files and roughly 20,300 lines, backed by 19 database tables and 27 recorded architectural decisions. Its shape is easy to describe: interfaces on the edge, a bridge that turns human messages into durable work, a continuity layer that assembles what the model sees, an intelligence layer that is replaceable, and a gate-and-evidence spine through which every effect must pass.

The graph below is the verified live path. Each arrow is code that runs in production today, exercised by the test suite and by the live probe; the diagram is not an aspiration.

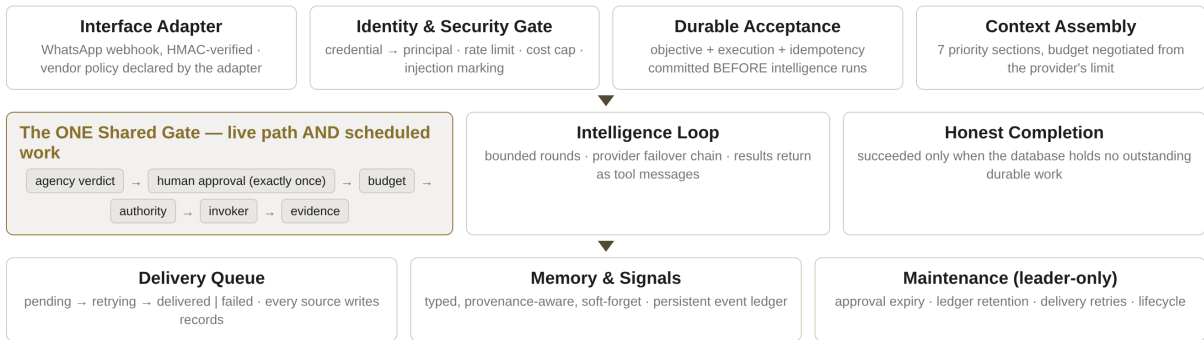


Figure 1. The verified runtime path, live and scheduled. One shared gate serves both.

Layer	What it owns	Status
Interface adapters	Vendor shapes, delivery policy, webhook verification. The core never sees vendor detail.	LIVE
Runtime bridge	Identity resolution, idempotency, security gate, context, the intelligence loop, honest completion.	LIVE
Durable work	Leases, fencing, retries, dead-letters, event- and time-wake scheduling.	LIVE
Delivery queue	Outbound messages as recoverable state: pending, retrying, delivered, failed.	LIVE
Maintenance	Leader-only sweeps: approvals, signals, deliveries, conversation lifecycle, reapers.	LIVE

Table 1. The runtime layers and their verified status.

4. Memory: The Living Substrate

Memory in WAX is per-principal, typed, and evidence-aware. Two doors create it: the bridge automatically records each exchange as an episodic memory, and the intelligence can store structured memories through a permission-gated capability. Records carry confidence, importance, provenance, observation time, and optional expiry. Typed links connect them: *supports*, *contradicts*, *derived_from*, *related_to*.

4.1 How memory evolves

New evidence can reinforce, weaken, contradict, or supersede an existing memory, and the history survives every transition. Supersession keeps the old record with a pointer to its replacement, so the system can answer both *what was believed* and *what is believed now*. Contradictions coexist as linked evidence instead of silently overwriting each other. Consolidation writes derived-from links before superseding sources, so a summary never destroys the evidence it came from.

4.2 Forgetting, stated honestly

Forgetting is real for access and soft for storage. A forgotten memory leaves every retrieval, search, link, and context path immediately, and the runtime's own reaper expires memories past their TTL. The row itself is retained for audit, which is the declared design. Hard deletion does not exist anywhere, and the sensitivity field on memory rows is an honest reserved placeholder until you set a privacy policy. Both boundaries are recorded in ADR-0025 rather than hidden.

5. Context: The Model's View of Reality

Context is not conversation history. It is the model's current operational view of reality, assembled from seven priority sections: the active objective first, then active and waiting work, recent conversation, relevant memories, artifacts, and environment facts. The assembly is budgeted: the runtime negotiates an evidence budget from the provider's advertised context limit, and lower-priority material compresses under pressure. The objective section cannot be crowded out by a long conversation, because it is priority zero and the conversation is priority two.

A session boundary is not a cognitive boundary. When someone disappears for days and returns, the runtime rebuilds the relevant state of reality from objectives, waiting work, memories, and artifacts rather than pretending the world began again. Every evidence line is labelled with its origin, so the question *why did this enter the model's context* is answerable from the execution trace.

6. Objectives and Continuity

An objective is an ongoing human intention, not a chat label. Every inbound message creates a durable objective and execution committed before any intelligence runs, which is what makes recovery possible: a crash leaves honest state that the recovery scan reconciles and a redelivered message can retry. One objective may produce many executions through retries and resumptions, and the history table records each participation.

Completion is evidence-driven. The runtime may only mark an objective succeeded when the database holds no outstanding durable work for it; if the interaction scheduled work, the objective stays waiting regardless of what the model says. Waiting states are evidence too: scheduled work makes the objective waiting, a pending human approval makes it awaiting-human, and consuming that approval reactivates it. This closed the most dangerous defect found in the previous cycle, where a tolerant sync could have fabricated a permanent, false terminal state.

7. Agentic Execution, Work and Time

The agent loop is generic: observe objective and environment, assemble context, reason, request capabilities through the gate, execute, observe results, record evidence, update memory and work state, and continue, wait, ask the human, or complete. It is not a planner feature; it is the execution cycle every request follows. The loop is bounded by round budgets, resource budgets, and timeouts, and every capability request passes through the same gate whether it comes from a live conversation or from scheduled work waking at 3 a.m.

Time is a primitive, not a feature. Durable work can wake at a time or on an event, correlated against a persistent signal ledger so work never misses a signal emitted before it was scheduled. Retries back off exponentially and are bounded; death is loud and recorded; requeue is the recovery path that preserves provenance. This is why a reminder is an emergent behavior: the runtime provides waiting, signals, delivery, and memory, and the intelligence composes them into whatever the human needs.

8. Capabilities and Environments

Twenty-one capabilities are registered, each one generic: durable work management, messaging through the human's own verified interfaces, memory operations, objective management, signal emission, approval inspection, isolated code execution, artifact acquisition, and ephemeral scratch workspaces. Discovery is live: every turn, the available capabilities and their contracts are presented to the model as tools. Extension is not: the registry is closed at boot by design, because the gates that would make third-party capabilities safe do not exist yet.

8.1 The one shared gate

The OMEGA cycle's most important structural change replaced two hand-maintained copies of the approval chain with a single shared component. The live bridge and the scheduled-work handler now run identical logic: the agency gate decides whether a human must authorize the action; an approved approval authorizes exactly one attempt of the exact same request (fingerprint-matched, consumed once, impossible to replay); and a pending approval is created atomically so two racing workers cannot multiply authority rows. The human is notified through their own interface, or, when no adapter is up, the notification becomes durable retry state instead of a silent log line.

Capability group	Examples	Status
Durable work	work.schedule, work.cancel, work.list, work.requeue	LIVE
Communication	message.send (identity-derived interface, delivery-policy aware)	LIVE
Memory	memory.store, memory.search, memory.forget, memory consolidate, memory.link	LIVE
Objectives	objective.list, objective.resume, objective.update_status	LIVE
Signals and approvals	signal.emit (runtime namespaces wait-only), approval.list, approval.cancel	LIVE
Compute and artifacts	code.run (namespace sandbox), workspace.acquire (integrity-pinned), scratch.workspace	LIVE
Open registry (acquisition of new capabilities)	designed lifecycle in ADR-0026; not implemented	DEFERRED
Media extraction on the live path	OCR and PDF extractors built and tested; not wired to inbound media	UNWIRED

Table 2. Capability inventory by group. Every capability passes the universal-primitive test.

8.2 Environments

Environment negotiation is real for what exists: the intelligence can request an ephemeral scratch workspace (time-limited, per-principal capped, leader-guarded reaper) and run code inside a kernel-level namespace sandbox with no network, a read-only root, masked system directories, resource limits, and loud fallback that never silently pretends to be stronger than it is. Container, microVM, and browser isolation grades are declared but raise an honest not-implemented error rather than faking security.

9. Interfaces: Adapters, Not Walls

WhatsApp is one interface, not the product. Everything vendor-specific lives in the adapter: webhook verification, message shaping, and the delivery policy the vendor imposes, such as the 24-hour customer-service window. The core runtime speaks only interface-neutral concepts, and architecture tests

scan the code so that vendor types cannot leak inward. A second interface, web or Telegram, is an adapter away: the same identity, memory, objectives, work, and continuity apply unchanged, and the approval decision grammar is deliberately generic so any adapter can adopt it in one call.

10. Security: The Runtime Enforces

The model is never the security boundary. Prompts carry state, not authority: even a fully compromised model output can only request capabilities, and every request passes identity, agency, approval, budget, and authority gates before anything happens. Identity is resolved from interface credentials by the runtime. The AI role has no permissions of its own. Ownership is enforced at the query level everywhere, so one principal's memory, work, or workspace is structurally invisible to another.

Guarantee	Enforced by	Evidence
Replay is impossible	Approvals consumed exactly once via a single conditional database update	Unit + concurrency tests
One pending approval per request	Partial unique index; creation is a single atomic statement	Migration f2b4d6a8c0e2 + tests
No self-authorization	Empty AI role; DB-backed role resolution on every invocation	Authority tests
No fabricated success	Terminal states require runtime evidence; outstanding work keeps objectives waiting	Objective lifecycle tests
No third-party messaging	Recipient must match the caller's own verified credential	Capability tests
Untrusted content stays data	Sanitizer marks injected content; gates hold regardless of model behavior	Security hardening suite
Cross-principal access refused	Query-level scoping plus per-call ownership checks	Hardening suite

Table 3. Core security guarantees and where they live.

Two boundaries are documented rather than faked. The rate, cost, and abuse defenses keep their counters in process memory, which means the runtime should be deployed as a single instance until the shared enforcement store is built (ADR-0027). And the namespace sandbox states its own limits honestly: it is strong isolation for accidents, not a hardened multi-tenant microVM.

11. Evaluation: How We Know

Unit tests alone are insufficient, so the runtime is evaluated scenario by scenario. The memory evaluation suite runs the full decathlon against real retrieval machinery: recall, irrelevance, update, temporal reasoning,

contradiction, abstention, privacy, forgetting, consolidation provenance, and retrieval poisoning. Thirteen open-world scenarios run objectives nobody hardcoded, through the real bridge, to prove the generic primitives compose for unknown domains. Thirty-two failure and recovery tests attack leases, fencing, provider failure, duplicates, partial execution, and delivery exhaustion.

The live probe exercises the actual production path over real HTTP, thirteen checks from webhook verification through a durable work item waking on an event, a destructive action being approved by the human over the live webhook grammar, and that action running exactly once. All thirteen pass. During this cycle the probe also demonstrated the new notification behavior live: when the vendor endpoint refused the approval notification, the message became durable retry state instead of a silent failure.

12. What Changed in the OMEGA Cycle

Four tracked defects, two races, and nine false claims were repaired, always with the same discipline: verify reality, design, implement, test, migrate, document, commit. The table shows the four you asked to track; each is a behavioral guarantee now, not a note in a register.

Tracked defect	Before	After
Duplicated approval creation	Two racing workers could create two pending approvals for the same request	Creation is one atomic statement against a partial unique index; the database decides, the loser returns the winner's row
Duplicated approval gate	Live path and scheduled work ran two drifting copies of the approval chain	One shared gate component; behavioral parity is tested, not assumed
Delivery recorded by one source	A failed send by the AI or by scheduled work was a dead letter	Every send failure becomes recoverable delivery state and reports queued-for-retry honestly
Silent background approvals	An approval created by scheduled work might never reach the human	The human is notified through a live channel, or the notification becomes durable retry state

Table 4. The four tracked items (register CV-11 to CV-14), before and after.

In the same batch: concurrent retries of the same message now claim the failed record with a conditional update so only one attempt can adopt it; the failure finalizer can no longer overwrite another attempt's in-flight state; the workspace reaper and the memory reaper run only on the elected leader; and a conversation lifecycle that advertised itself as a background task now actually runs in one. The false claims corrected ranged from a memory state no writer ever produced (removed) to a ledger described as append-only that is really retention-pruned (reworded).

13. Current Limitations and Honest Boundaries

Eighteen of the nineteen database tables grow without any pruning. This is deliberate and documented: retention is a policy decision about people's data, and ADR-0025 hands you a decision sheet instead of inventing rules. Until a policy lands, nothing is deleted, and every claim in the code says exactly what actually happens.

- **Single-instance enforcement perimeter.** Rate, cost, and abuse defenses are in-memory; running multiple instances would multiply every limit. The durable work core is already multi-instance correct; the enforcement perimeter is not (ADR-0027).
- **No hard deletion.** Forgetting is access-revocable soft delete; erasure awaits your policy (ADR-0025).
- **Capability registry stays closed.** Open acquisition needs provenance, signing, and registration approval first; the build order is designed in ADR-0026.
- **Output schemas are declared, not validated;** the idempotency key on capability requests is accepted but not yet honored (ADR-0026).
- **Media extraction and account deletion are unwired:** the code exists and is tested, but no live path uses them yet.
- **Rolling multi-instance deploys** would fail each other's long executions; the execution-heartbeat fix is designed and deferred (ADR-0027).

These are the boundaries the mission asked to be surfaced rather than smoothed over. None of them is hidden behind a misleading name, a hopeful comment, or a demo.

14. Future Expansion Boundaries

The runtime is ready for more without structural change. New capabilities slot into the registry and are immediately schedulable, approvable, budgetable, and auditable. New interfaces attach as adapters without touching the cognitive architecture. New model providers join the failover chain through configuration. When you decide the retention policy, the pruner implements your decision rather than inventing one. When a real need arrives that the registry cannot serve, ADR-0026 is the build order for opening it safely: persistent descriptor storage, provenance and security metadata, output validation, registration approval, and a verify-wrap-register pipeline for acquired capabilities.

The shortest honest summary of the system remains the one at the heart of the mission: the intelligence chooses, the infrastructure enables, the runtime enforces, evidence proves, memory preserves, context informs, and time continues. This cycle made every one of those clauses a verified property of the code, and made the places where that is not yet true explicit by name.